

**TEKNOFEST**

**HAVACILIK, UZAY VE TEKNOLOJİ  
FESTİVALİ**

**ULAŞIMDA YAPAY ZEKA YARIŞMASI**

**KRİTİK TASARIM RAPORU**

**TAKIM ADI**

**AVION AI**

**BAŞVURU ID**

**381923**

## İÇİNDEKİLER

|                                                 |    |
|-------------------------------------------------|----|
| İÇİNDEKİLER.....                                | 2  |
| ŞEKİL LİSTESİ, TABLO LİSTESİ, KISALTMALAR ..... | 3  |
| 1-TAKİM ŞEMASI .....                            | 4  |
| 2-PROJE MEVCUT DURUM DEĞERLENDİRMESİ .....      | 4  |
| 2.1-YAPILAN DEĞİŞİKLİKLER .....                 | 4  |
| 2.2-MEVCUT DURUM DEĞERLENDİRMESİ.....           | 5  |
| 3-ALGORİTMA VE SİSTEM MİMARİSİ.....             | 5  |
| 3.1-VERİ SETLERİ.....                           | 5  |
| 3.2-ALGORİTMALAR .....                          | 7  |
| 4-ÖZGÜNLÜK .....                                | 8  |
| 5-SONUÇLAR VE İNCELEME.....                     | 9  |
| 6-KAYNAKÇA .....                                | 11 |



## ŞEKİL LİSTESİ

|                                                                                                               |    |
|---------------------------------------------------------------------------------------------------------------|----|
| Şekil 1:Takım Üyeleri ve Görev Paylaşımı .....                                                                | 4  |
| Şekil 2:Örnek Eğitim Videosu .....                                                                            | 6  |
| Şekil 3:Devrim Veri Seti.....                                                                                 | 6  |
| Şekil 4:Stanford Drone Veri Seti.....                                                                         | 7  |
| Şekil 5: DenseNet ve CSPNet hız kıyaslaması [6]      Şekil 6:DenseNet ve CSPNet backbone kıyaslaması[6] ..... | 8  |
| Şekil 7:Deney 3 Confusion Matrix      Şekil 8:Deney 5 Confusion Matrix.....                                   | 9  |
| Şekil 9:Deney 4 Model Tahmini.....                                                                            | 10 |
| Şekil 10:Deney 1 Model Tahmini.....                                                                           | 10 |

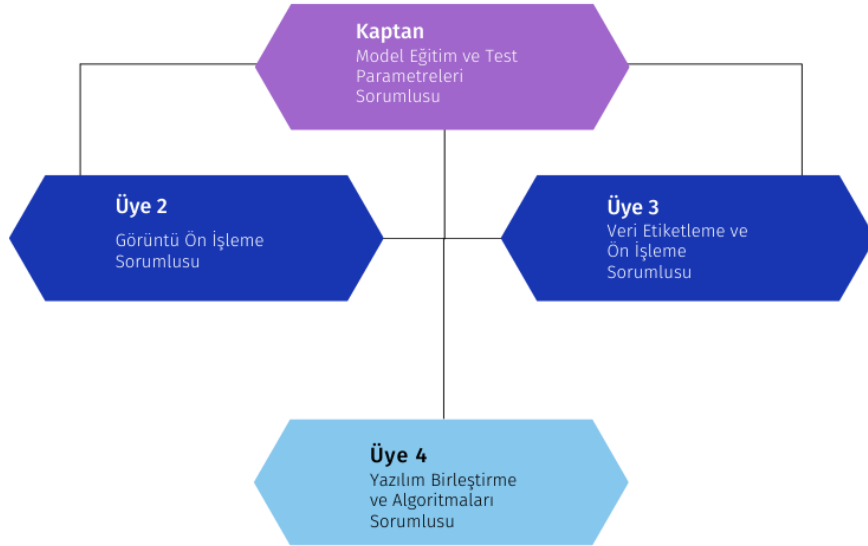
## TABLO LİSTESİ

|                                     |   |
|-------------------------------------|---|
| Tablo 1:Deneyler ve Sonuçları ..... | 9 |
|-------------------------------------|---|

## KISALTMALAR

- YOLO: You Only Look Once
- YOLT: You Only Look Twice
- FP: False Positive
- CNN: Convolutional Neural Network (Evrşimsel Sinir Ağı)
- mAP: Metrics Average Precision
- PR: Precision-Recall
- ROC: Receiver Operator Characteristic
- GAN: Generative Adversarial Network
- BFLOPS: Billion Floating Point Operations Per Second [Saniyede Bir Yapılan Float İşlemleri (Milyar)]

## 1-TAKIM ŞEMASI



Şekil 1: Takım Üyeleri ve Görev Paylaşımı

## 2-PROJE MEVCUT DURUM DEĞERLENDİRMESİ

### 2.1-Yapılan Değişiklikler

Ön Tasarım Raporu aşamasında yarışma tarafından sağlanan ve kendi oluşturacağımız veri setlerine ek olarak VisDrone ve Stanford veri setlerinin kullanılması planlanmaktayken, sonrasında yaptığımız incelemelerde VisDrone veri setinin çekim açısının yarışmanın amacına hizmet etmediği, Stanford veri setinin ise bizim veri setimizdeki dengesizlik oranını artırdığı görüldü. Bu sorunu çözmek için bir yöntem geliştirildi, uygulaması üzerinde ise çalışmalarımız sürmekte. Bu yöntem kapsamında insan ve taşıt gibi fazla bulunan objelerin yoğunluğunu azaltmak için eleme işlemi yapıp dengelenmiş veriyi veri setine dahil ederek veri setimizin genişliğini artırmayı hedeflemekteyiz.

Algoritmalar açısından ise, ön tasarım raporunda belirtildiği üzere, nesne tanıma modelimiz olan YOLOv5 [1] algoritmasından aldığımız nesne çıktılarını bir görüntü sınıflandırma modeli ile güçlendirip modelimizin sınıflandırma başarı oranını artırmak için çalışmalarımızı ve testlerimizi yaptık. Bu sistem için kurduğumuz mimarinin ve bu modellerin ortalama çalışma süreleri hesaba katıldığında, tespit edilmesi gereken nesne sayısının çok olması durumunda yarışma anında bize verilen süreyi aşma ihtimalimizin olduğunu gördük. Bu sebeple bu algoritmayı yarışmanın diğer bir görevi olan iniş alanlarının doluluk tespiti aşamasında kullanma kararı aldık. Bu kapsamda nesne tespit modelimizin UAP ve UAİ olarak tespit ettiği bounding box'lar kırılıp dolu ve boş sınıflandırması yapan bir CNN modeline verilecek. Bu CNN modelinin eğitimi için veri seti oluşturma işlemlerimiz sürüyor.

## 2.2-Mevcut Durum Değerlendirmesi

Ön Tasarım Raporu aşamasında yaptığımız araştırmalar, kritik tasarım raporu hazırlama sürecinde bizlere rehberlik etti. Proje takviminde de planlandığı üzere, veri seti hazırlama çalışmalarımızda ilerleme kaydettik. Yarışma tarafından verilen veri setinin üzerine kendi drone çekimlerimizi ekleyerek öncelikle veri havuzumuzu genişlettik. Sonrasında ise etiketleme çalışmalarımızı tamamlayarak veri setlerini birleştirme işlemini yaptık. Bu süreçte videoların birer görüntü setine dönüşmesi, etiketleme ve eğitim/validasyon/test setlerine ayrılması gibi süreçlerden geçtik. Şu aşamada ise farklı ön işlemlerden geçmiş veriler ile modelimizi eğiterek sonuçları inceliyoruz. Bu bağlamda histogram eşitleme ve tiling gibi yöntemler denenip sınıflandırma ve mAP değerlerindeki etkileri gözlemleniyor.

Bunlara ek olarak, veri setimizde bulunan fotoğraf ve obje sayıları dikkate alındığında, çeşitli veri çoğaltma yöntemlerine ihtiyaç duyduğumuzu fark ettik. Ayrıca UAP ve UAİ sayılarının insan ve taşıt sayılarına kıyasla çok az olması dengesiz bir dağılıma neden oluyordu. Bu sorunları çözmek için geliştirdiğimiz mimari kapsamında veri setimizi dengeledik, yüksek bir başarı oranı için gereken sayıya getirdik. Bu aşamada x ve y eksenlerinde aynalama, blurlama, döndürme vb. yöntemler kullanıldı.

Veri ekibimiz bu işlemleri yaparken, model ekibimiz araştırma ve deneylerini sürdürdü. YOLOv5 modelinin parametrelerinin başarı oranı, hız gibi sonuçlara etkilerini incelemek için öncelikle metrik yorumlama çalışmaları yapıldı. PR eğrisi, ROC eğrisi, F1 skoru ve mAP değerlerinin yorumlanması, nasıl değerler alması gerektiği tartışıldı, dokümanite edildi. Bu süreçte öğrendiklerimizi uygulamak adına VisDrone, Stanford gibi kaynaklardan alınmış veri setleri ile model eğitip sonuçları gözlemledik. Kendi veri setimiz hazır olduğunda ise araştırma aşamasında öğrendiklerimizi uygulayarak model eğitime ve metrik yorumlama çalışmalarına başladık. Geldiğimiz son aşamada ise bu metrikler arasından mAP değerinin artırılması için parametre optimizasyonu ve transfer öğrenme yöntemleri inceleniyor.

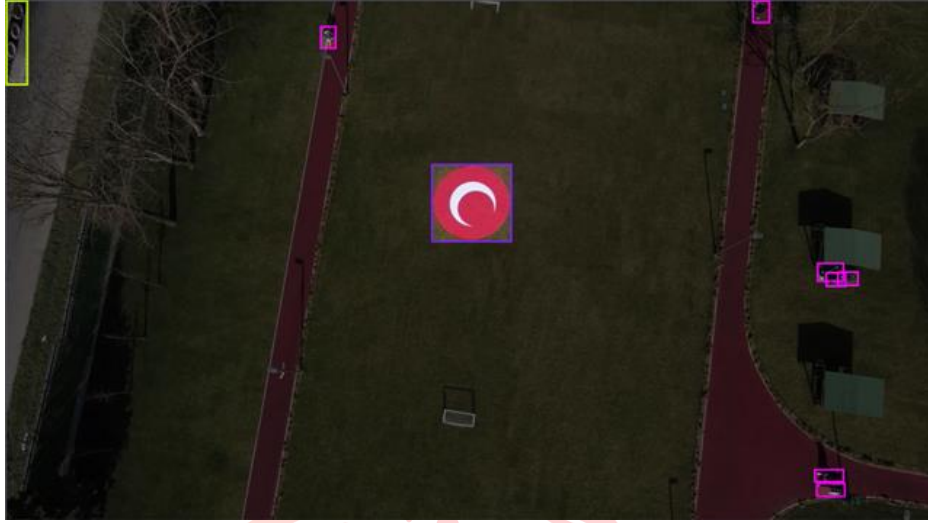
Bu aşamaların yanında, veri seti ve model ekibimizin ortak çalışmaları sonucunda modelimizin küçük nesneleri tespit etmekte başarısız olduğu anlaşıldı. Bunu çözmek için incelenen yöntemler arasında tespit (inference) aşamasında sliding uygulaması, YOLO'nun YOLT [2] gibi özelleştirilmiş versiyonları ve benzeri uygulamalar bulunmaktadır. Sliding yaparak tespit yapma metodunun tespit (inference) süresinin optimizasyonu üzerinde çalışıyoruz. Ek olarak YOLTv5 modeli ile de çalışmalarımız sürmekte olup bu modelin büyük nesneleri tespit etmedeki başarısını incelemekteyiz.

## 3-ALGORİTMALAR VE SİSTEM MİMARİSİ

### 3.1-Veri Setleri

#### a-)Örnek Eğitim Videosu

Modelimizin eğitiminde öncelikle yarışma tarafından sağlanan örnek eğitim videosu kullanılmıştır. Bu videodan yaklaşık 1450 görüntü elde edildi ve bu görüntüler Şekil-2'de görüleceği gibi teker teker MakeSenseAi kullanılarak işaretlendi. İşaretlerken "insan, tasit, uai, uap" adlı sınıflara ayırdık.



Şekil 2:Örnek Eğitim Videosu

Yarışma puanlamasında modelimizin mAP değeri baz alınacaktır. Bu sebepten dolayı framerleri işaretlerken bounding box'ların doğruluğu ve isabetliliğine ayrıca dikkat edilmiştir. Bu videodan elde ettiğimiz veri değerlendirildiğinde sınıflar arasında dengesizlik olduğu ve “uai, uap” sınıflarından çok az sayıda örnek varken oldukça fazla “tasit” sınıfından örnek görülmüştür. Bu dengesizliği düzeltmek ve veri çeşitliliğini artırmak amacıyla yeni veri seti arayışına geçilmiştir.

#### b-)Devrim Veri Seti

İnternet üzerinde istediğimiz açı ve yükseklikten çekilmiş bir veri seti olmadığından ve daha fazla uai uap verisine ihtiyaç duyduğumuzdan kendi veri setimizi oluşturduk. Bunun için belirtilen ölçü ve renklerde uai ve uap alanlarının fiziksel örneği oluşturuldu. Ardından belirtilen yükseklik ve açılarda Orta Doğu Teknik Üniversitesi Devrim Stadyumu'nda drone ile çekim yapılarak oldukça zengin bir veri seti oluşturuldu.Bu veri setinden 1500 kare elde edildi ve bunların şimdilik 600 adedi Şekil-3' teki gibi etiketlenerek veri setine dahil edildi.



Şekil 3:Devrim Veri Seti

### c-)Stanford Drone Veri Seti

Araştırmalarımız sonucunda çekim açısı ve niteliği bizim formatımıza en çok uyan veri setinin halka açık paylaşılan Stanford Drone Veri Seti olduğuna karar verdik. Bu veriler insan, bisiklet, kaykay sürücüsü gibi etiketlere sahipti. 8 farklı bağlamda 60 farklı video içeren bu veri setinin etiketleri nesne takibi amacıyla oluşturulmuş metin dosyalarından oluşmaktadır. Bu videolardan 30 karede bir görüntü alarak ve her 30 kare için metin dosyalarından gerekli etiketleri YOLO formatına dönüştürerek yaklaşık 9000 adet etiketlenmiş görüntü veri elde ettik. Ayrıca etiketleri de bizim modelimizin sınıflarına uyacak şekilde düzenledik. Ancak bu etiketleri ve verileri kontrol ettiğimizde Şekil 4'te görüldüğü gibi bounding boxlardaki bozukluk ve sınıflar arasındaki aşırı dengesizlikten dolayı şimdilik bu veri setini daha sonraki süreçlerde ihtiyaç dahilinde kullanmaya karar verdik.



Şekil 4:Stanford Drone Veri Seti

### 3.2-Algoritmalar

Problem çözümünde temel olarak YOLO algoritması üzerinden gidiyoruz ancak YOLO'nun, bu problemin çözümü için tek başına yeterli bir araç olmadığını ön tasarım raporunda belirtmiştik. Bu sebeple YOLO üzerine kurulmuş ancak küçük değişiklikler yapılmış bir ağ üzerinden testlerimizi sürdürmekteyiz. YOLT (You Only Look Twice) [2] YOLO üzerinden türetilmiş, uzay ve hava görüntüleri kullanılarak eğitilmiş bir modeldir. YOLT modeli küçük nesneleri tanımda daha başarılı olduğu için eğitimlerimize YOLO/YOLT birlikte çalıştırıp karşılaştırma yaparak ilerliyoruz. Bunun dışında YOLO'nun bazı zayıf yönleri ve bizim çözümlerimiz şunlardır:

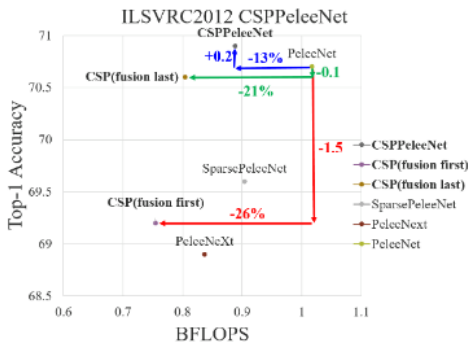
- Modelin nesnelerin genellemesinde veya eğitildiğinden farklı açılarda karşısına çıkması durumunda zorlanması  
**Çözüm:** Rescaling ve Rotation resim çoğaltma teknikleri
- Kuş sürüsü gibi küçük objelerin tespitinde zorlanması
- YOLO'nun birden fazla downsampling layer kullandığı için obje tespitinde görece biraz daha vurdumduymaz çalışıyor olması  
**Çözüm:**
  - Yeni bir network(YOLT) üzerinde daha yoğun bir final katmanı kullanmak
  - Daha önceden YOLO ağı üzerinde sıkıştırılarak (down-sampled) incelenen objelerde küçük obje tespitini efektif kılmak için bu yoğun katmanları tekrar netleştirmek (up-sample)
  - Farklı ölçeklerde tespit edicilerin çalıştırılarak hem küçük hem de büyük obje tespitini efektif kılmak

Yukarıda da bahsedildiği üzere YOLT veya YOLO üzerinden objeleri yüksek bir doğrulukla tespit ettikten sonra problemi tam anlamıyla çözebilmek için UAP ve UAİ alanlarının iniş durumlarının uygunluklarını sınıflandırmamız gerekiyor. Ön tasarım raporumuzda UAP ve UAİ alanlarının iniş durumlarının uygunluğunu ayrı bir CNN modelinden geçirmeyi uygun görmüştük. Bu yüzden de öncelikle temel bir CNN inşa ederek bunun üzerinden çalışıyoruz. UAP ve UAİ alanlarını tespit edip YOLO/YOLT üzerinden elde ettiğimiz etiketlerle alanları kırpıp tekrar CNN'e gönderiyoruz. Ancak çözünürlükte düşüş yaşadığımız için farklı resim zenginleştirme ve görüntü işleme teknikleri kullanarak modelin başarısını artırmayı düşünüyoruz. Clahe, Copy-Pasting [3], Rotating, Rescaling şu anda üzerinde çalıştığımız tekniklerdir.

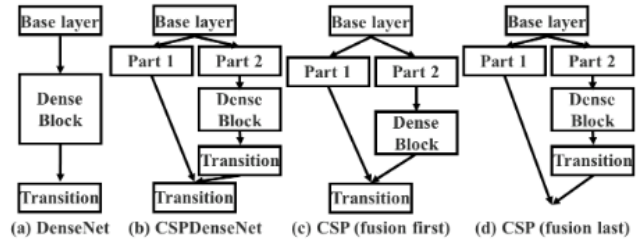
#### 4.ÖZGÜNLÜK

Yarışma tarafından bize sağlanan görüntülere ek olarak UAP ve UAİ görsellerini brandalara gerçek ölçüde bastırıp drone çekimi yaparak elde ettiğimiz görüntüleri de modelimizi eğtmek için kullandığımız veri setimize ekledik. Bu görüntüler hem yüksek çözünürlüklü hem de yarışma şartlarına uygun olmaları sebebiyle bizim çalışmamıza ve bu alanda yapılacak sonraki çalışmalara temel olabilecek niteliktedir. Bunun dışında yarışma formatına uyacak şekilde farklı hava durumları içeren görüntüler üretmek için GAN [4] [5] kullanma niyetimizden ön tasarım raporumuzda bahsetmiştik. Modelimiz şu anda elimizde olan görüntüler üzerinde belirli bir performansa ulaştıktan sonra bu görüntüleri de oluşturup modelimizi kötü hava şartlarından etkilenmeyecek niteliğe erdirmek ilerideki planlarımız arasındadır. Bunların dışında veri setimizi genişletmeyi kolaylaştırmak için yeni ekleyeceğimiz resimleri etiketleme işini modelimizin bir önceki sürümüne yaptırmaktayız. Modelimizin oluşturacağı etiketler üzerinde düzeltmeler yapmak sıfırdan etiketlemeye kıyasla kolaylık oluşturmakta ve iş yükünü azaltıp bize zaman kazandırmaktadır. Bu sayede bir veri seti oluştururken en büyük zorluk olan etiketleme aşamasında harcanan zamanı azaltarak veri setini genişletmemiz daha kolay hale gelmiştir.

Ön tasarım raporunda da daha önce belirttiğimiz gibi YOLO'nun Backbone'u üzerinde değişiklik yapmayı planlıyorduk. Kullanabileceğimiz alternatiflerden biri olan CSPNet, denkları ResNet, ResNeXt ve DesNet'e göre hesaplama işlemlerini %20 daha azaltıyor [6]. Şekil 5 ve 6 CSPNet'in çeşitli versiyonlarının alternatifleriyle karşılaştırmasını göstermektedir. Görüldüğü üzere CSP(Fusion Last) bunlar içerisinde en ideal çözümü veriyor, bu nedenle biz de modelde kullandığımız Backbone'u oluştururken CSPNet'in bu versiyonunu temel aldık. Ancak modelimizin daha hızlı çalışması amacıyla bu versiyon üzerinde küçük bir değişiklik yaparak modelimizi oluşturduk. Bizim yaptığımız değişiklik CSPNet'in girdi alırken girdileri ikiye ayırma mekanizmasını değiştirerek iki Convolutional katman kullanmak şeklindedir. Bu iki yöntem teoride aynı hızda çalışsa da testlerimizde bizim yöntemimizin daha hızlı çalıştığını gördüğümüz için bu yöntemi kullanma kararı aldık.



Şekil 5: DenseNet ve CSPNet hız kıyaslaması [6]



Şekil 6: DenseNet ve CSPNet backbone kıyaslaması [6]

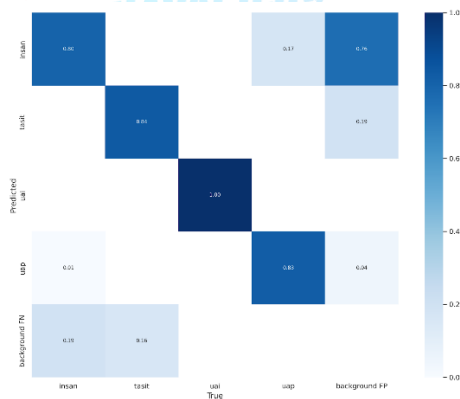
## 5.SONUÇLAR VE İNCELEME

Çeşitli değişkenlerin sistemimiz üzerindeki etkilerini görmek amacıyla kontrollü deneyler yaptık. Kullanılan algoritmayı, modele giren resim boyutunu ve başka birtakım değişkenleri değiştirdiğimiz deneylerin özeti Tablo-1'de verilmiştir.

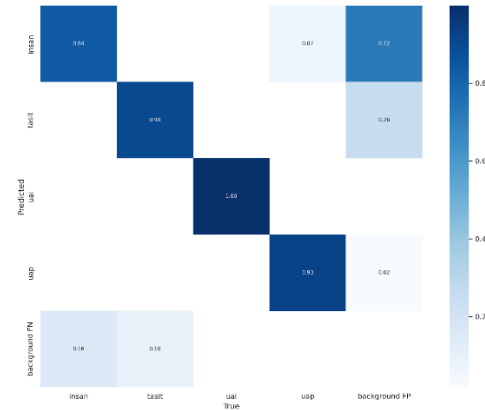
| Deney no | Kullanılan Algoritma | Resim boyutu | Değiştirilen parametreler | Veri setindeki görüntü sayısı | Epoch Sayısı | Precision | Recall | mAP 0.5 | mAP 0.5:0.9 |
|----------|----------------------|--------------|---------------------------|-------------------------------|--------------|-----------|--------|---------|-------------|
| 1        | YOLTV5               | 1280         | copy-paste=0              | 1450                          | 99           | 0.9683    | 0.9663 | 0.9760  | 0.7409      |
| 2        | YOLOv5               | 1280         | copy-paste=0.1            | 2100                          | 99           | 0.9141    | 0.8773 | 0.9054  | 0.6670      |
| 3        | YOLOv5               | 1280         | copy-paste=0              | 2100                          | 99           | 0.9251    | 0.8959 | 0.9013  | 0.6444      |
| 4        | YOLOv5               | 640          | copy-paste=0              | 2100                          | 99           | 0.9128    | 0.8533 | 0.8875  | 0.6345      |
| 5        | YOLTV5               | 1280         | copy-paste=0              | 2100                          | 99           | 0.9133    | 0.9016 | 0.9185  | 0.6641      |
| 6        | YOLOv5               | 1280         | Hyperparameter-Evolution  | 2100                          | 99           | 0.9263    | 0.8433 | 0.904   | 0.6651      |

Tablo 1:Deneyler ve Sonuçları

- Deney #1 - Deney #3 - Deney #5: YOLO ve YOLT algoritmalarını kıyasladık ve YOLT'un YOLO'ya göre daha başarılı olduğunu gördük ancak bu başarının veri setine bağlı olup olmadığını başka testlerle de inceledik. Bkz. [Şekil 7-8] Ayrıca Deney #3 ve #5 e bakıldığında mAP\_0.5:0.9 skorunun YOLT'ta aynı veri setinde daha yüksek olduğunu gördük.
- Deney #2 - Deney #3: Küçük nesnelerin tespiti için Copy-Paste algoritmasının etkisini araştırdık ve Copy-Paste aktifken daha iyi sonuç aldık.
- Deney #3 - Deney #4: Modele sokulan görüntü çözünürlüğünü test ettik ve çözünürlük daha yüksek olduğunda daha iyi sonuç aldığımızı fark ettik.

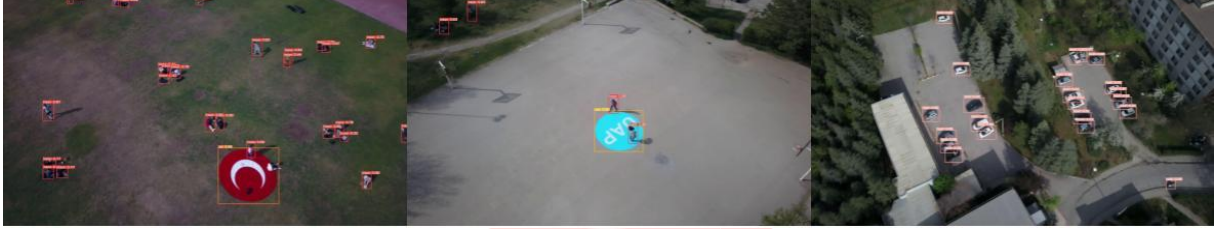


Şekil 7:Deney 3 Confusion Matrix



Şekil 8:Deney 5 Confusion Matrix

Şekil 9 ve 10: Şekil 5, YOLO modelinin (Deney #3) ürettiği sonuç iken Şekil 10, YOLT (Deney #5) modelinin ürettiği sonuç ve sınıflandırma konusunda YOLT'un daha başarılı olduğunu görebiliyoruz. Aynı zamanda burada tespit ettiğimiz bir sorun Background FP'lerin insan sınıfında yüksek olması. Bunun temel sebebi insan sınıfı diğer objelere kıyasla çok daha küçük objelerden oluştuğu için bazen insan olmayan objelere de insan sınıflandırması yapıyor. Bunu yeni teknikler araştırarak ve veri setimizi genişleterek artırmak niyetindeyiz.



Şekil 9:Deney 4 Model Tahmini



Şekil 10:Deney 1 Model Tahmini

Şekil 9 ve Şekil 10'da görüldüğü üzere YOLT, YOLO'ya göre başarısız oldu ancak daha önce test ettiğimiz metriklerle göre başarılı olmasını bekliyorduk. Burada en büyük hatamız veri seti değişkenini deney yaparken sabit tutmamak oldu. Model mevcut veri seti üzerinde çok iyi başarı gösterse de küçük insan gruplarının etiketli fotoğrafı Deney #1 yapılırken elimizde yoktu. Bu gecikme bizim deney sonuçlarını ilk başta yanlış yorumlamamıza sebep oldu. Ancak Deney #5 verilerinden de görüldüğü üzere aynı veri setinde YOLT daha iyi sonuç verdi.

Ayrıca Hyperparameter Evolution yani modelin Hyperparameter değerlerini de başka bir modelden geçirerek yarattığımız öğrenme süreci Deney #6'da YOLO modeli üzerinden test edilmiştir. Hyperparameter Evolution yapılmadığı (Deney #3) duruma göre model mAP\_05:0.9 skorunda 0.02'lik bir artış göstermiştir.

Veri setimizi genişleterek ve bu kontrollü deneylere devam ederek problemi tam anlamıyla çözebilmek için en uygun çözümü üretmeye devam ediyoruz.

Deneylerimizi yapmak için Google Colab servisi üzerinden NVIDIA Tesla P100 grafik kartlarını kullanıyoruz. Bu sayede tespit yaparken bir kareyi 0,5 saniyeden daha kısa bir sürede işleyebiliyoruz. Yarışmada kullanmak üzere aynı kapasitede grafik kartları kullanarak benzer bir donanım kurma çalışmalarımız şu an devam etmektedir.

## 6.KAYNAKÇA

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2015). You Only Look Once: Unified, Real-Time Object Detection. arXiv. <https://doi.org/10.48550/ARXIV.1506.02640>
- [2] Van Etten, A. (2018). *You Only Look Twice: Rapid Multi-Scale Object Detection In Satellite Imagery*. doi:10.48550/ARXIV.1805.09512
- [3] Kisantal, M., Wojna, Z., Murawski, J., Naruniec, J., & Cho, K. (2019). *Augmentation for small object detection*. <https://arxiv.org/pdf/1902.07296.pdf>.
- [4] Li, Xuelong & Kou, Kai & Zhao, Bin. (2021). Weather GAN: Multi-Domain Weather Translation Using Generative Adversarial Networks. <https://doi.org/10.48550/arXiv.2103.05422>
- [5] Tremblay, M., Halder, S. S., de Charette, R., & Lalonde, J. F. (2020). Rain Rendering for Evaluating and Improving Robustness to Bad Weather. *International Journal of Computer Vision*, 129(2), 341–360. <https://doi.org/10.1007/s11263-020-01366-3>. Açık kaynak kodları: <https://github.com/cv-rits/rain-rendering>
- [6] Wang, C.-Y., Liao, H.-Y., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., & Yeh, I-Hau. (n.d.). CSPNet: A New Backbone that can Enhance Learning Capability of CNN. Retrieved June 14, 2022, from [https://openaccess.thecvf.com/content\\_CVPRW\\_2020/papers/w28/Wang\\_CSPNet\\_A\\_New\\_Backbone\\_That\\_Can\\_Enhance\\_Learning\\_Capability\\_of\\_CVPRW\\_2020](https://openaccess.thecvf.com/content_CVPRW_2020/papers/w28/Wang_CSPNet_A_New_Backbone_That_Can_Enhance_Learning_Capability_of_CVPRW_2020)

